

ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Semestrálna práca z predmetu

Algoritmy a údajové štruktúry 2

SEMESTRÁLNA PRÁCA 1, ZADANIE 2

Vypracoval: Juraj Nižnanský

Študijná skupina: 5ZIB11

Akademický rok: 2026/2027

V Žiline dňa 23.10.2026

Obsah

1. Quad Strom implementovaný v semestrálnej práci 2
2. KD Strom implementovaný v semestrálnej práci 3
3. Quad Strom spojený s KD Stromom implementovaný v semestrálnej práci 4
4. Quad Strom z GitHub repozitára používateľa Varunpant 5
5. KD Strom z GitHub repozitára od používateľa Oschrenk 6

1. Quad Strom implementovaný v semestrálnej práci

Pri testovaní Quad stromu boli využité celkovo 4 testy:

1. **Rýchlostný test operácie vloženia:** meranie času, za ktorý sa dáta zo zadaného súboru ‚slovakia_nodes.csv' vložili do štruktúry pomocou Apache NetBeans profilera
2. **Rýchlostný test hľadania:** meranie času, za ktorý sa vyhľadali všetky zadané oblasti zo zadaného súboru ‚testovacie_oblasti.csv' pomocou Apache NetBeans profilera
3. **Test správnosti operácie vloženia:** do stromu bolo vložených 100000 náhodných hodnôt (x, y, dáta). Kontrola, či sa všetky vložené hodnoty v strome nachádzali.
4. **Test správnosti operácie hľadania:** nad stromom boli s pravdepodobnosťou 50% robené operácie vloženia a hľadania 100000-krát (t.j. približne 50000 operácií vloženia a 50000 operácií hľadania). Kontrola, či sa pri hľadaní náhodných oblastí našli všetky požadované hodnoty.

Výsledky testovania:

1. **Test:** rýchlosť operácie vloženia bola približne 200 ms
2. **Test:** rýchlosť operácií hľadania bola približne 900 ms



3. **Test:** všetky vložené hodnoty sa v strome nachádzali
4. **Test:** všetky operácie hľadania našli správne hodnoty

2. KD Strom implementovaný v semestrálnej práci

Pri testovaní Quad stromu boli využité celkovo 4 testy:

1. Rýchlostný test operácie vloženia: meranie času, za ktorý sa dáta zo zadaného súboru ‚slovakia_nodes.csv' vložili do štruktúry pomocou Apache NetBeans profilera

2. Rýchlostný test hľadania: meranie času, za ktorý sa vyhľadali všetky zadané oblasti zo zadaného súboru ‚testovacie_oblasti.csv' pomocou Apache NetBeans profilera

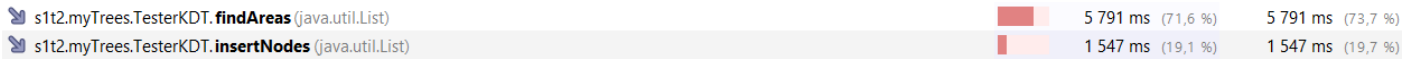
3. Test správnosti operácie vloženia: do stromu bolo vložených 100000 náhodných hodnôt (x, y, dáta). Kontrola, či sa všetky vložené hodnoty v strome nachádzali.

4. Test správnosti operácie hľadania: nad stromom boli s pravdepodobnosťou 50% robené operácie vloženia a hľadania 100000-krát (t.j. približne 50000 operácií vloženia a 50000 operácií hľadania). Kontrola, či sa pri hľadaní náhodných oblastí našli všetky požadované hodnoty.

Výsledky testovania:

1. Test: rýchlosť operácie vloženia bola približne 1500 ms

2. Test: rýchlosť operácií hľadania bola približne 5800 ms



3. Test: všetky vložené hodnoty sa v strome nachádzali

4. Test: všetky operácie hľadania našli správne hodnoty

3. Quad Strom spojený s KD Stromom implementovaný v semestrálnej práci

Pri testovaní Quad stromu boli využité celkovo 4 testy:

1. Rýchlostný test operácie vloženia: meranie času, za ktorý sa dáta zo zadaného súboru ‚slovakia_nodes.csv' vložili do štruktúry pomocou Apache NetBeans profilera

2. Rýchlostný test hľadania: meranie času, za ktorý sa vyhľadali všetky zadané oblasti zo zadaného súboru ‚testovacie_oblasti.csv' pomocou Apache NetBeans profilera

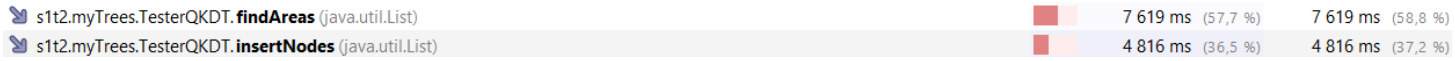
3. Test správnosti operácie vloženia: do stromu bolo vložených 100000 náhodných hodnôt (x, y, dáta). Kontrola, či sa všetky vložené hodnoty v strome nachádzali.

4. Test správnosti operácie hľadania: nad stromom boli s pravdepodobnosťou 50% robené operácie vloženia a hľadania 100000-krát (t.j. približne 50000 operácií vloženia a 50000 operácií hľadania). Kontrola, či sa pri hľadaní náhodných oblastí našli všetky požadované hodnoty.

Výsledky testovania:

1. Test: rýchlosť operácie vloženia bola približne 5000 ms

2. Test: rýchlosť operácií hľadania bola približne 7500 ms



3. Test: všetky vložené hodnoty sa v strome nachádzali

4. Test: všetky operácie hľadania našli správne hodnoty

4. Quad Strom z GitHub repozitára používateľa Varunpant

Pri testovaní Quad stromu boli využité celkovo 4 testy:

1. Rýchlostný test operácie vloženia: meranie času, za ktorý sa dáta zo zadaného súboru ‚slovakia_nodes.csv' vložili do štruktúry pomocou Apache NetBeans profilera

2. Rýchlostný test hľadania: meranie času, za ktorý sa vyhľadali všetky zadané oblasti zo zadaného súboru ‚testovacie_oblasti.csv' pomocou Apache NetBeans profilera

3. Test správnosti operácie vloženia: do stromu bolo vložených 100000 náhodných hodnôt (x, y, dáta). Kontrola, či sa všetky vložené hodnoty v strome nachádzali.

4. Test správnosti operácie hľadania: nad stromom boli s pravdepodobnosťou 50% robené operácie vloženia a hľadania 100000-krát (t.j. približne 50000 operácií vloženia a 50000 operácií hľadania). Kontrola, či sa pri hľadaní náhodných oblastí našli všetky požadované hodnoty.

Výsledky testovania:

1. Test: rýchlosť operácie vloženia bola približne 600 ms

2. Test: rýchlosť operácií hľadania bola približne 4000 ms



3. Test: všetky vložené hodnoty sa v strome nachádzali

4. Test: všetky operácie hľadania našli správne hodnoty

5. KD Strom z GitHub repozitára od používateľa Oschrenk

Pri testovaní KD stromu boli využité celkovo 4 testy:

1. **Rýchlostný test operácie vloženia:** meranie času, za ktorý sa dáta zo zadaného súboru ‚slovakia_nodes.csv' vložili do štruktúry pomocou Apache NetBeans profilera
2. **Rýchlostný test hľadania:** meranie času, za ktorý sa vyhľadali všetky zadané oblasti zo zadaného súboru ‚testovacie_oblasti.csv' pomocou Apache NetBeans profilera
3. **Test správnosti operácie vloženia:** do stromu bolo vložených 100000 náhodných hodnôt (x, y, dáta). Kontrola, či sa všetky vložené hodnoty v strome nachádzali.
4. **Test správnosti operácie hľadania:** nad stromom boli s pravdepodobnosťou 50% robené operácie vloženia a hľadania 100000-krát (t.j. približne 50000 operácií vloženia a 50000 operácií hľadania). Kontrola, či sa pri hľadaní náhodných oblastí našli všetky požadované hodnoty.

Výsledky testovania:

1. **Test:** rýchlosť operácie vloženia bola približne 300 ms
2. **Test:** rýchlosť operácií hľadania bola približne 4000 ms



3. **Test:** všetky vložené hodnoty sa v strome nachádzali
4. **Test:** všetky operácie hľadania našli správne hodnoty

6. Problémy pri používaní stiahnutých štruktúr

Pri používaní stiahnutých štruktúr boli zistené tieto problémy:

- **Implementácia od používateľa Oschrenk:** umožňuje používať iba double súradnice. Nie je umožnená univerzálnosť kľúča.
- **Implementácia od používateľa Varunpant:** umožňuje používať iba double súradnice. Nie je umožnená univerzálnosť kľúča.

7. Intervalové vyhľadávanie v KD Strome

Algoritmus intervalového vyhľadávania v KD strome, implementovanom v semestrálnej práci:

```
public LinkedList<T> find(Comparable[] minKeys, Comparable[] maxKeys) {
    // Exchange keys if min > max
    for (int i = 0; i < this._k; i++) {
        if (minKeys[i].compare(maxKeys[i]) > 0) {
            Comparable temp = minKeys[i];
            minKeys[i] = maxKeys[i];
            maxKeys[i] = temp;
        }
    }

    LinkedList<T> found = new LinkedList<>();
    if (this._root == null) {
        return found;
    }

    // Stacks
    LinkedList<KDNode<T>> nodeStack = new LinkedList<>();
    LinkedList<Integer> depthStack = new LinkedList<>();

    nodeStack.push(this._root);
    depthStack.push( e: 0);
```

```
    while (!nodeStack.isEmpty()) {
        KDNode<T> current = nodeStack.pop();
        int depth = depthStack.pop();
        int keyIndex = depth % this._k;

        // Left Subtree MAX < Key
        if (maxKeys[keyIndex].compare(current.getKey(keyIndex)) == -1) {
            KDNode<T> leftSon = current.getLeftSon();
            if (leftSon != null) {
                nodeStack.push(leftSon);
                depthStack.push( e: depth + 1);
            }
        }
        // Right Subtree MIN > Key
        else if (minKeys[keyIndex].compare(current.getKey(keyIndex)) == 1) {
            KDNode<T> rightSon = current.getRightSon();
            if (rightSon != null) {
                nodeStack.push(rightSon);
                depthStack.push( e: depth + 1);
            }
        }
```

```
        // Both Subtrees Key: <MIN, MAX>
        else {
            if (this.nodeInInterval(current, minKeys, maxKeys)) {
                found.add(current.getData());
            }

            KDNode<T> rightSon = current.getRightSon();
            if (rightSon != null) {
                nodeStack.push(rightSon);
                depthStack.push( e: depth + 1);
            }
            KDNode<T> leftSon = current.getLeftSon();
            if (leftSon != null) {
                nodeStack.push(leftSon);
                depthStack.push( e: depth + 1);
            }
        }
    }

    return found;
}
```

1. **Výmena kľúčov:** v prípade, že minimálne kľúče sú menšie ako maximálne, tak sa vymenia
2. **Kontrola koreňa:** pokiaľ strom nemá koreň (t.j. je prázdny), vráti sa prázdny zoznam
3. **Inicializácia zásobníkov:**
    a. Do zásobníka s uzlami sa vkladajú tie, ktoré bude potrebné prehľadať
    b. Do zásobníka s hĺbkami sa vkladajú príslušné hĺbky, na ktorých sa uzol nachádza (potrebné pre porovnávanie správnych kľúčov)
4. **Vloženie koreňa do zásobníka spolu s hĺbkou 0**
5. **Cyklus while:**
    a. Pokiaľ nie je zásobník s uzlami prázdny, algoritmus sa opakuje
    b. Zo zásobníkov sa vyberie posledný vložený uzol spolu s príslušnou hĺbkou
6. **Prehliadka aktuálneho uzla v cykle while:**
    a. **Maximálny hľadaný kľúč < kľúč aktuálneho uzla:**
        i. Je potrebné prehľadať ľavý podstrom, v ktorom sú menšie kľúče
        ii. Do zásobníka sa vloží ľavý syn (pokiaľ existuje)
    b. **Minimálny hľadaný kľúč > kľúč aktuálneho uzla:**
        i. Je potrebné prehľadať pravý podstrom, v ktorom sú väčšie kľúče
        ii. Do zásobníka sa vloží pravý syn (pokiaľ existuje)
    c. **Kľúč aktuálneho uzla** $\in <$ ***Minimálny hľadaný kľúč, Maximálny hľadaný kľúč*** $>$**:**
        i. Je potrebné prehľadať ľavý aj pravý podstrom
        ii. Do zásobníka sa vloží pravý aj ľavý syn (pokiaľ existujú)
        iii. **Kontrola všetkých kľúčov aktuálneho uzla:**
            1. Pokiaľ pre všetky kľúče platí, že patria do hľadaného intervalu, aktuálny uzol je vložený do zoznamu nájdených uzlov
7. **Po skončení cyklu sa vráti zoznam dát nájdených uzlov**

Pozn.: Algoritmus v momentálnej podobe pravdepodobne nefunguje efektívne. Zatiaľ som neprišiel na spôsob, ako tento algoritmus zastaviť skôr ako v listoch stromu. Algoritmus však nevyužíva rekurziu a neprehľadáva celý strom – pri vyhľadávaní sú niektoré podstromy vynechané.